Lab: Splitting Up the Work

COM-3820 Parallel Programming

Count the words in every file of a dataset in parallel, and find out how the way you split the work into tasks decides the speedup.

Setup

Install uv if needed, then:

```
uv sync                 # installs Python 3.14 if needed
uv python install 3.14t # a second Python without the GIL, for Part 1
uv run gen_small.py     # data/small: 2,000 files of 10-100 KB
uv run gen_mixed.py     # data/mixed: 10,000 files of 1-4 KB, plus two of 60 and 100 MB
uv run run.py small     # times the sequential version and saves its answer
uv run run.py mixed
```

You write your code in `lab.py`. `run.py` times it and checks its answer; `labutil.py` has helpers. Always use `uv run`, never plain `python`.

Record your results and answers in `answers.html` (open it in a browser): paste each `run.py` output into its box and answer the questions there. Its Save button downloads the JSON file you submit.

Your numbers will differ from your classmates': they depend on your machine's number of cores, the kind of cores, and the operating system. `run.py` prints your CPU count at the top of every run. Answer every question for your own machine, and explain your numbers in terms of it.

Part 1: Threads, one file per task

Write `count_threads`, then compare thread counts:

```
uv run run.py small --mode threads --workers 1,2,4,8,16
```

- Does it get faster with more threads? Why not?

Now run the same code on Python 3.14t, a build of Python without the GIL (`run.py` shows `GIL off`):

```
uv run --isolated --python 3.14t run.py small --mode threads --workers 1,2,4,8,16
```

- What changed, and what does that tell you about the first run?
- Compare the 1-thread times of the two runs. What does removing the GIL cost?

Part 2: Processes, one file per task

Write `count_procs`, then compare pool sizes:

```
uv run run.py small --mode procs --workers 1,2,4,8,16
```

- How does the speedup grow with the pool size? Where does it stop growing, and how does that compare with your CPU count?

Part 3: Many tiny files and a few huge ones

Run your Part 2 code on the mixed dataset:

```
uv run run.py mixed --mode procs --workers 4,8,16
```

- Why does it barely beat sequential, whatever the pool size?

Now write `count_tasks`, where each task is about `task_bytes` of text, whatever the file boundaries. Then try different task sizes:

```
uv run run.py mixed --mode tasks --task-kb 4,64,1024,16384,65536
```

- Think about the two extremes in this dataset. Is a 2 KB file worth a task of its own? Should one worker get a whole 100 MB file while the others wait?
- Which task size is fastest? Why are the smallest and the largest slow?
- `labutil.read_range(path, start, end)` reads part of a file without cutting words in two.

Last step: sub-interpreters

Python 3.14 adds `InterpreterPoolExecutor`: its workers are threads in one process, but each thread runs its own Python interpreter with its own GIL, and they share no objects. In `count_procs` and `count_tasks`, replace `ProcessPoolExecutor` with `InterpreterPoolExecutor` and run again:

```
uv run run.py small --mode procs --workers 1,2,4,8,16
uv run run.py mixed --mode tasks --task-kb 4,64,1024,16384,65536
```

- How do your numbers compare with processes, and with the free-threaded threads in Part 1? What do sub-interpreters have in common with processes?

Submission

Check your `lab.py` and your saved answers JSON file into the `./lab-2` folder of the **PP-COM3820** branch that you created for this course in the YU GitHub system.